

ADR-014 — Architecture multi-société, multi-domaine, multi-fournisseur

Table des matières

Résumé	3
Glossaire	3
Contexte	4
Le cas d'usage réel	4
Ce que l'architecture actuelle n'exprime pas	4
Décision	5
Les quatre entités du modèle	5
Les relations	6
Multi-OS : beryl n'est pas cantonné à FreeBSD	6
Capabilités par fournisseur	7
Organisation des fichiers	8
~/.beryl/ — arborescence proposée	8
.env.yml — trois niveaux	8
<domaine>.yml — structure	9
_account.yml — métadonnées société (optionnel)	9
Architecture du code en couches	9
Polymorphisme des providers	10
Les commandes CLI	11
Refonte immédiate (cet ADR)	11
Commandes existantes (inchangées côté signature)	11
Commandes de listing (nouvelles, priorité basse)	11
Horizon (pas dans cet ADR, documenté pour cohérence)	11
Flux typique d'un nouveau setup	12
Migration depuis l'état actuel	12
Plan de commits	12
Conséquences	13
Positives	13
Négatives / coûts	13
Neutres	14
Évolutions futures (hors scope de cet ADR, mais prévues)	14
Multi-admin : plusieurs opérateurs partagent la configuration	14
Pistes techniques	14
Ce que l'architecture proposée permet déjà	14
Ce qui manquera et qu'il faudra ajouter	15
Coffre-fort pour les clés SSH	15
Pistes techniques	15
Ce que l'architecture proposée permet déjà	15
Note de sécurité	15
Alternatives considérées	16
A. Rester sur le modèle « un domaine = un fournisseur »	16

Résumé

Beryl doit gérer le cas réel d'un opérateur qui pilote des serveurs pour plusieurs sociétés, chacune ayant ses propres comptes chez plusieurs fournisseurs, et dont les domaines peuvent être répartis entre ces fournisseurs (zone DNS chez l'un, hébergement chez l'autre). L'architecture actuelle part du fournisseur comme unité de configuration. Cet ADR propose de partir de la **société** et de traiter explicitement les rôles distincts « gestion DNS », « hébergement », et plus tard « stockage objet » / « certificats ».

Quatre axes structurent cette vision :

1. Multi-société — modéliser explicitement le payeur qui détient les comptes fournisseurs.
2. Multi-fournisseur par société — un compte par fournisseur, avec ses propres credentials, sans duplication entre les domaines de la société.
3. Multi-OS — beryl ne doit pas rester cantonné à FreeBSD. L'OS est un attribut du host et le flux bootstrap/apply est polymorphique.
4. Multi-admin et coffre-fort (horizon) — plusieurs opérateurs partagent la même configuration via un stockage de credentials abstrait (Vault, SOPS, 1Password...).

L'objectif est un beryl **clair pour un nouvel utilisateur** qui découvre l'outil, quel que soit son fournisseur, sa société ou son OS.

Glossaire

Pour un nouvel utilisateur de beryl, les termes suivants reviennent partout. Cet ADR les utilise au sens strict défini ici :

Société (code : Account)	Entité morale qui paie les comptes chez les fournisseurs. Une société a typiquement une personne contact, éventuellement un numéro client chez chaque fournisseur. Dans beryl, c'est l'unité de regroupement de tout le reste : comptes, domaines, serveurs.
Fournisseur (code : Provider)	Service cloud consommé par beryl : OVH, Scaleway, Gandi, Cloudflare, Hetzner, AWS... Chaque fournisseur a un nom court (ovh) et déclare ses capacités (qu'il sait faire du DNS, de l'hébergement compute, du stockage objet...).
Capability (code : symbole Crystal, ex : :dns)	Rôle technique qu'un fournisseur sait remplir. Beryl en connaît plusieurs : :dns, :compute, :object_storage, :cdn, :cert. Un même fournisseur peut avoir plusieurs capacités (OVH fait DNS + compute). Beryl vérifie la capability avant d'appeler une méthode.

Compte fournisseur (code : ProviderAccount)	Couple (société, fournisseur) + les credentials associés. Une société a typiquement un ProviderAccount par fournisseur qu'elle utilise. Les credentials vivent dans ~/.beryl/.env.yml à l'intersection [société][fournisseur].
Domaine (code : Domain)	Zone DNS (ex : aloli.net). Rattachée à une société. Déclare un dns_provider (qui gère la zone) et un default_compute_provider (hébergeur par défaut des serveurs).
DNS provider	Le fournisseur qui héberge la zone DNS du domaine, et qui accepte les records A/AAAA/CNAME/PTR. Pour aloli.net aujourd'hui : OVH. Pourrait être Gandi, Cloudflare, Route53 demain.
Compute provider	Le fournisseur qui héberge les serveurs physiques (ou virtuels). Peut différer du DNS provider : aloli.fr peut avoir son DNS chez Gandi et ses serveurs chez OVH.
Host / Serveur (code : Host)	Machine (physique ou virtuelle) pilotée par beryl. Rattachée à un domaine, optionnellement à un groupe. Peut surcharger le compute_provider du domaine.
OS (code : Host#os)	Système d'exploitation cible du host. Aujourd'hui freebsd. Prévu : debian, ubuntu, alpine, rocky...
Groupe (code : Group)	Ensemble de hosts partageant la même config au sein d'un domaine (ex : web/rails01, web/rails02). Factorise la config.
Bootstrap	Opération d'installation complète d'un OS sur un host neuf (wipe disques, partitionnement, base system, users, packages, sudoers).
Apply	Opération de mise en conformité d'un host existant avec sa config YAML (sync packages, users, clés SSH, sudoers, services...).

Contexte

Le cas d'usage réel

Philippe Nénert (Aloli) gère :

- aloli.net — domaine payé par la société ALOLI avec un compte OVH (domain registrar + DNS + hébergement). Des serveurs sous ce domaine sont prévus chez OVH et Scaleway.
- aloli.fr — domaine payé par la société ALOLI mais chez Gandi (registrar + DNS). Serveurs prévus chez OVH et Scaleway, factures au compte ALOLI de ces fournisseurs.
- quimeo.net — domaine payé par la société QUIMEO (cliente) avec un autre compte OVH. Serveurs QUIMEO chez OVH et Scaleway, credentials API totalement distincts de ceux d'ALOLI.

Ce que l'architecture actuelle n'exprime pas

L'architecture actuelle de beryl (23 avril 2026, branche `production`) tient dans :

```
# ~/.beryl/aloli.net.yml
provider: ovh
ovh:
  ssh_key_name: philippe-aloli-fr
ssh_keys:
  - philippe.aloli.fr.pub
```

yaml

```
# ~/.beryl/.env.yml
aloli.net:
  OVH_APPLICATION_KEY: xxx
  OVH_APPLICATION_SECRET: yyy
  OVH_CONSUMER_KEY: zzz
```

yaml

Problèmes :

1. `.env.yml` est indexé par domaine, pas par société. Si ALOLI a deux domaines (`aloli.net` + `aloli.fr`) avec les mêmes credentials OVH, on est forcé de les répéter.
2. Le champ `provider:` est ambigu : est-ce le DNS ? L'hébergeur ? Les deux (cas implicite OVH uniquement) ?
3. Impossible d'exprimer « DNS chez Gandi, hébergement chez OVH » sans hardcoder OVH dans `beryl scan --dns`.
4. `beryl init` part du fournisseur, ce qui n'est pas la bonne unité de départ — on ne lance pas `beryl` pour « configurer OVH », on le lance pour « gérer les domaines d'une société ».

Décision

Les quatre entités du modèle

Account (société)	Payeur. Une société a un jeu de credentials par fournisseur.	nom court : <code>aloli</code> , <code>quimeo</code>
Provider	Fournisseur de services cloud. Déclare ses capabilities (<code>:dns</code> , <code>:compute</code> , <code>:object_storage</code> ...). Peut être présent ou absent du build de <code>beryl</code> .	nom court : <code>ovh</code> , <code>scaleway</code> , <code>gandi</code> , <code>cloudflare</code> ...
Domain (zone DNS)	Rattaché à une société, avec un <code>dns_provider</code> (qui gère la zone) et un <code>default_compute_provider</code> (hébergeur par défaut).	FQDN de la zone : <code>aloli.net</code>

Host (serveur)	Rattaché à un domaine (via un éventuel groupe). Peut surcharger le <code>compute_provider</code> . Déclare son OS (FreeBSD aujourd'hui, autres dans le futur).	nom court : loulou ; FQDN : loulou.aloli.net
----------------	---	--

Les relations

Account (société payeuse)

- └─ N ProviderAccount – un compte par fournisseur
- └─ N Domain – les zones DNS gérées
- └─ N Host – via Domain (parfois via Group)

ProviderAccount (Account × Provider + credentials)

- └─ belongs_to: Account
- └─ belongs_to: Provider (via nom)
- └─ credentials – dans `.env.yml[account][provider]`

Provider (entrée du catalogue beryl)

- └─ name – ovh, scaleway, gandi...
- └─ capabilities – Array(Symbol)
- └─ implemented? – true/false dans ce build

Domain (zone DNS)

- └─ belongs_to: Account
- └─ dns_provider – qui héberge la zone (1 seul)
- └─ default_compute_provider – hébergeur par défaut des hosts
- └─ N Host

Host (serveur)

- └─ belongs_to: Domain
- └─ belongs_to: Group? – optionnel
- └─ compute_provider? – override du default Domain
- └─ os – freebsd, debian, ubuntu...
- └─ freebsd_config / debian_config / ...

Multi-OS : beryl n'est pas cantonné à FreeBSD

Le beryl actuel est construit autour de FreeBSD (mfsBSD+bsdinstall, `pkg`, `/usr/local/etc`, shells `csh/zsh` par défaut). L'architecture cible pose l'OS comme **attribut du host**, avec une section de config dédiée par OS :

```
# host FreeBSD (aujourd'hui)
os: freebsd
freebsd:
  install_type: distribution_sets
  packages: [sudo, zsh, git]
  zfs:
    zroot: { boot: true, raid: 0, disks: [/dev/sda] }
```

yaml

```
# host Debian (vision future)
os: debian
debian:
  release: bookworm
  packages: [sudo, zsh, git, ufw]
  partitioning:
    scheme: lvm
    disks: [/dev/sda]
```

yaml

```
# host Alpine, Rocky, Ubuntu...
os: alpine
alpine:
  release: "3.20"
  packages: [sudo, bash, openssh]
```

yaml

L'OS résout automatiquement :

- Bootstrap : FreeBSD via mfsBSD-in-QEMU ; Linux via cloud-init / debootstrap / PXE (selon le fournisseur). Chaque provider de compute propose ses voies d'install disponibles.
- Package manager : pkg pour FreeBSD, apt pour Debian/Ubuntu, apk pour Alpine, dnf pour Rocky...
- Paths système : /usr/local/etc vs /etc, /home/admin/.zshrc localisation...
- Shells par défaut : /bin/csh FreeBSD, /bin/bash Debian, /bin/ash Alpine.

Les actions `rescue`, `scan`, `bootstrap`, `apply` deviennent polymorphiques par OS (pattern matching simple sur `host.os`). Un refactor léger côté beryl suffit aujourd'hui ; l'implémentation Linux viendra quand le besoin se présentera, sans casser ce qui tourne.

Côté code :

```
abstract class Beryl::OsBootstrapper
  abstract def bootstrap(host : Host) : Nil
end

class Beryl::Os::FreeBsd < Beryl::OsBootstrapper
  def bootstrap(host) : Nil
    # chemin actuel : mfsBSD-in-QEMU + bsdinstall
  end
end

class Beryl::Os::Debian < Beryl::OsBootstrapper
  def bootstrap(host) : Nil
    # futur : debootstrap ou cloud-init
  end
end
```

crystal

Capabilities par fournisseur

Un `Provider` déclare ce qu'il sait faire. Beryl vérifie ces capabilities avant d'appeler une méthode :

OVH	:dns, :compute	□ partiel
Scaleway	:compute (futur : :dns, :object_storage)	□ partiel
Gandi	:dns	□ à venir
Cloudflare	:dns, :cdn	□ futur
Hetzner	:compute	□ futur
AWS	:compute, :object_storage, :dns	□ futur

Un `Domain` qui pointe vers un provider sans la capability nécessaire (ex : `dns_provider: hetzner` alors qu'Hetzner n'a pas `:dns`) lève une erreur explicite à la résolution.

Organisation des fichiers

~/.beryl/ — arborescence proposée

```
~/.beryl/
├── _default.yml           # socle technique commun à toutes sociétés
├── .env.yml               # credentials – 3 niveaux
├── aloli/                 # dossier société
│   ├── _account.yml      # métadonnées société (optionnel)
│   ├── aloli.net.yml     # domaine
│   ├── aloli.net/
│   │   ├── loulou.yml    # host direct
│   │   └── web/          # groupe
│   │       ├── _group.yml # config du groupe
│   │       ├── rails01.yml
│   │       └── rails02.yml
│   └── aloli.fr.yml
└── quimeo/
    ├── quimeo.net.yml
    ├── quimeo.net/
    └── web.yml
```

La société est **dérivée du chemin** : `~/.beryl/aloli/aloli.net.yml` appartient forcément à la société `aloli`. Pas besoin d'un champ `account` dans chaque fichier domaine.

.env.yml — trois niveaux

```
aloli:
  ovh:
    OVH_APPLICATION_KEY: xxx
    OVH_APPLICATION_SECRET: yyy
    OVH_CONSUMER_KEY: zzz
```

yaml

```

scaleway:
  SCW_SECRET_KEY: abc
gandi:
  GANDI_API_KEY: def          # quand Gandi sera implémenté

quimeo:
  ovh:
    OVH_APPLICATION_KEY: aaa  # compte OVH distinct
    OVH_APPLICATION_SECRET: bbb
    OVH_CONSUMER_KEY: ccc
  scaleway:
    SCW_SECRET_KEY: ddd

```

Un domaine lit ses credentials via [] [], quel que soit le nombre de domaines de la société. Zéro répétition.

<domaine>.yml — structure

```

# ~/.beryl/aloli/aloli.net.yml
dns_provider: ovh          # qui gère la zone aloli.net
provider: ovh              # hébergeur par défaut (= default_compute_provider)

ovh:
  ssh_key_name: philippe-aloli-fr

scaleway:                  # facultatif : si plusieurs hosts Scaleway dans ce domaine
  ssh_key_ids: [uuid-philippe]

ssh_keys:
  - philippe.aloli.fr.pub

```

yaml

```

# ~/.beryl/aloli/aloli.fr.yml
dns_provider: gandi        # DNS chez Gandi
provider: ovh              # hébergeur par défaut chez OVH (même compte aloli)

ovh:
  ssh_key_name: philippe-aloli-fr

ssh_keys:
  - philippe.aloli.fr.pub

```

yaml

_account.yml — métadonnées société (optionnel)

```

# ~/.beryl/aloli/_account.yml
name: ALOLI SAS
contact_email: contact@aloli.fr
notes: |
  Facturation interne. Contact principal : Philippe.

```

yaml

Si absent, beryl infère juste `name = aloli` (nom du dossier).

Architecture du code en couches

CLI	(rescue, bootstrap, scan, apply, wipe, init, ...)
Actions	(orchestrent les appels aux providers) RescueAction, BootstrapAction, DnsSetupAction, ...
Resolver	(FQDN → Host avec merge hiérarchique)
Model	Account / Domain / Host / Group / Provider
Credentials Store	(.env.yml abstrait derrière une API)
Provider Registry	(catalogue + capabilities + plugins)
Config Loader	(lit ~/.beryl/<account>/...)

Chaque couche a une responsabilité unique. Les **Actions** n'ont pas à savoir où sont stockés les credentials : elles demandent au `CredentialsStore` via `store.for(account, provider)`. Le **Loader** n'a pas à connaître les providers : il charge du YAML en suivant une hiérarchie de chemins.

Polymorphisme des providers

Un `Provider` est une classe qui `include` des modules correspondant à ses capabilities :

```
abstract class Beryl::Provider
  abstract def name : String
  abstract def capabilities : Array(Symbol)
  abstract def implemented? : Bool
end

module Beryl::DnsProvider          # pour capability :dns
  abstract def list_records(zone : String) : Array(DnsRecord)
  abstract def ensure_record(zone, type, sub_domain, target)
  abstract def set_reverse(ip, fqdn)
  abstract def refresh(zone)
end

module Beryl::ComputeProvider      # pour capability :compute
  abstract def rescue(host)
  abstract def boot_from_disk(host)
  abstract def list_ssh_keys : Array(SshKeyInfo)
end

class Beryl::Providers::Ovh < Beryl::Provider
  include Beryl::DnsProvider
  include Beryl::ComputeProvider
  # ...
end
```

crystal

Un shard tiers (ex : `crystal-beryl-gandi`) n'a qu'à :

1. Créer class `Gandi < Beryl::Provider`; `include Beryl::DnsProvider`; `end`
2. Appeler `Beryl::Providers.register(Gandi.new)`

Beryl le découvre au démarrage via le registre.

Les commandes CLI

Refonte immédiate (cet ADR)

<code>beryl init <société></code>	Crée <code>~/.beryl/<société>/</code> . Propose d'ajouter providers et domaines en chaîne.	<code>beryl init aloli</code>
<code>beryl add-provider <provider> [--account=S]</code>	Ajoute un fournisseur à une société. Credentials écrits dans <code>.env.yml[account][provider]</code> . Pour OVH : auto-gen consumer key avec routes complètes.	<code>beryl add-provider scaleway --account=aloli</code>
<code>beryl add-domain <domaine> [--account=S]</code>	Crée <code>~/.beryl/<société>/<domaine>.yaml</code> . Demande DNS provider, default compute provider, SSH key.	<code>beryl add-domain aloli.net --account=aloli</code>

Si une seule société existe dans `~/.beryl/`, `--account` est optionnel (auto-déecté). S'il y en a plusieurs, beryl demande laquelle.

Commandes existantes (inchangées côté signature)

- `beryl rescue <host>`
- `beryl scan <host> [--dns --write]`
- `beryl bootstrap <host>`
- `beryl apply <host>`
- `beryl wipe <host>`
- `beryl boot-hd <host>`
- `beryl show <host>`
- `beryl list-hosts`

En interne, elles passent par le nouveau modèle (Account/Domain/Host) et lisent les credentials via le nouveau canal.

Commandes de listing (nouvelles, priorité basse)

- `beryl list-accounts`
- `beryl list-providers [--account=S]`
- `beryl list-domains [--account=S]`

Horizon (pas dans cet ADR, documenté pour cohérence)

- `beryl info <host>` — métadonnées serveur côté provider
- `beryl ssh <host>` — raccourci
- `beryl run <host> <cmd>` — exécution distante
- `beryl dns list/add/remove --domain=X`
- `beryl cert request/renew`
- `beryl backup list/create/restore`

Flux typique d'un nouveau setup

```
$ beryl init aloli
[beryl init] Société « aloli » créée dans ~/.beryl/aloli/
[beryl init] Ajouter un fournisseur maintenant ? [o/N] : o
# ... chaîne vers add-provider ...

$ beryl add-provider ovh --account=aloli
# prompt app_key + app_secret
# auto-gen consumer key (POST /auth/credential)
# écrit .env.yml[aloli][ovh]

$ beryl add-provider scaleway --account=aloli
# prompt SCW_SECRET_KEY
# écrit .env.yml[aloli][scaleway]

$ beryl add-domain aloli.net --account=aloli
# Qui gère la zone DNS ? [ovh, scaleway] : ovh
# Hébergeur par défaut ? [ovh, scaleway] : ovh
# Sélection SSH key OVH... ↔ ~/.ssh/philippe.aloli.fr.pub
# Écrit ~/.beryl/aloli/aloli.net.yml
```

console

Migration depuis l'état actuel

Philippe a confirmé le 23 avril 2026 : « Que cela casse quoi que ce soit, ça m'est égal, nous ne sommes pas en production. » Pas de migration automatique. Le `~/.beryl/` actuel sera refait manuellement avec les nouvelles commandes `init` / `add-provider` / `add-domain`.

Côté tests : les fixtures `spec/fixtures/config/` sont toutes réorganisées vers la nouvelle arborescence. Chaque fixture utilisée par un spec devient explicite sur sa société.

Plan de commits

Chantier pensé en **9 commits** sur `production`, avec la CI au vert à chaque étape. Les premiers commits posent le modèle sans toucher aux CLI ; les CLI migrent ensuite vers ce modèle ; le flux `init` est refondu en dernier pour consommer toute l'infrastructure.

1. Model — Account, Domain, Host, Group, ProviderAccount comme vraies classes. Tests unitaires isolés.
2. Provider Registry — capabilities, implemented?, interfaces DnsProvider / ComputeProvider.
3. Credentials Store — .env.yml à 3 niveaux, API store.for(account, provider).
4. Config Loader — scan de ~/.beryl/<account>/<domain>.yml sous-dossiers hosts/groupes.
5. Resolver — FQDN → Host avec merge hiérarchique (default + domain + group + host).
6. Providers Refactor — OVH implémente DnsProvider ComputeProvider. Scaleway implémente ComputeProvider.
7. CLI Actions — rescue, bootstrap, scan, apply, wipe, boot-hd utilisent les capabilities au lieu de host.provider codé en dur.
8. Nouveau flux init — init, add-provider, add-domain. Retrait de l'ancien beryl init <provider>.
9. Fixtures + documentation — specs migrés, README mis à jour.

À chaque commit :

- crystal spec au vert.
- Build réussi (crystal build).
- CI GitHub au vert.
- Message de commit en français, court, qui explique le pourquoi.

Conséquences

Positives

- Zéro duplication des credentials quand plusieurs domaines sont payés par la même société chez le même fournisseur.
- Séparation explicite des rôles DNS / Compute / (futur) Storage / Cert, via le système de capabilities.
- Extensibilité tierce : un fournisseur = un shard Crystal qui implémente les bonnes interfaces, plug-and-play dans beryl.
- Flux init conforme au modèle mental : on part de la société (entité métier), pas du fournisseur (détail d'implémentation).

Négatives / coûts

- Refactor structurant : 9 commits, touche la plupart des fichiers.
- Casse la config locale de Philippe : il devra refaire son ~/.beryl/ depuis zéro (acté).

- Complexité conceptuelle augmentée : Account / Domain / Host / Provider / ProviderAccount / DnsProvider / ComputeProvider, ça fait beaucoup d'entités. Documentation soignée nécessaire.

Neutres

- Le YAML utilisateur reste lisible — la complexité du modèle est dans le code, pas dans les fichiers de config.
- Les commandes existantes (rescue, bootstrap...) gardent la même signature CLI.

Évolutions futures (hors scope de cet ADR, mais prévues)

Ces deux sujets ne sont **pas** implémentés dans les 9 commits listés plus haut. L'architecture proposée doit cependant les **permettre** sans avoir à tout repenser.

Multi-admin : plusieurs opérateurs partagent la configuration

Aujourd'hui, `~/ .beryl/` vit sur la machine d'un seul utilisateur. Dès qu'il y a une équipe, plusieurs admins doivent pouvoir :

- lancer beryl rescue, bootstrap, apply sur les mêmes serveurs ;
- partager la liste des sociétés, domaines, hosts (le modèle lui-même) ;
- partager les credentials API, sans les envoyer par email ou Slack ;
- suivre qui a fait quoi (trace d'audit minimum).

Pistes techniques

1. Dépôt git privé par équipe — l'arborescence `~/ .beryl/` (sauf `.env.yml`) vit dans un repo privé. Les admins clonent, pull régulièrement, commit leurs changements. Bénéfice : versioning gratuit, audit via git log.
2. Fichier `.env.yml` chiffré — avec age, sops, ou git-crypt. Chaque admin a sa clé, le fichier chiffré vit dans le git avec le reste.
3. Vault externe — HashiCorp Vault, AWS Secrets Manager, Infisical, Bitwarden Secrets Manager. Beryl ne stocke rien en local, il interroge le vault à chaque exécution.
4. Password manager CLI — 1Password, Bitwarden, KeePassXC ont des CLIs qui exposent les secrets à la demande. Beryl passe par le CLI.
5. Granularité des droits — un admin peut avoir accès à la société ALOLI et pas QUIMEO. Implémenté côté stockage (dossier git séparé, path vault restreint...) plutôt que côté beryl.

Ce que l'architecture proposée permet déjà

- CredentialsStore abstrait : la couche credentials est derrière une interface `store.for(account, provider)`. L'implémentation actuelle lit `.env.yml plaintext`. Une future implémentation peut lire depuis `age/sops/vault/1password` sans que les CLI ou Actions s'en rendent compte.
- Arborecence `~/beryl/` : organisable comme un repo git, chaque société dans son dossier. Commitable tel quel (hors `.env.yml`).
- Actions idempotentes : un admin peut relancer sans crainte de corruption, les ressources (DNS records, reverses, CK OVH) sont gérées via `ensure_*` plutôt que `create_*` aveugle.

Ce qui manquera et qu'il faudra ajouter

- Un mécanisme de lock pour éviter que deux admins lancent un bootstrap sur le même host simultanément. Pas urgent.
- Un journal d'audit local (qui a fait quoi quand). Probablement une ligne par commande dans `~/beryl/.history` ou équivalent.
- Une commande `beryl sync` pour pull/push vers le git partagé.

Coffre-fort pour les clés SSH

Aujourd'hui les clés SSH privées vivent dans `~/ssh/`. Beryl les **référence** par nom de fichier (`philippe.aloli.fr.pub`) et résout à la lecture. Bien pour un opérateur seul ; limitant en équipe.

Pistes techniques

1. ssh-agent classique — pas de clé en clair sur le disque. Chaque admin a sa clé en agent, beryl utilise ssh qui passe par `SSH_AUTH_SOCK`. Marche déjà aujourd'hui si l'opérateur configure son agent.
2. YubiKey / carte à puce (PIV, OpenPGP, FIDO2) — la clé ne sort jamais du token. Beryl utilise ssh qui s'interface avec le hardware via PKCS#11 ou ssh-agent adapté.
3. Signing SSH Certificates — une autorité de certification SSH interne signe des certificats courts (24h) pour les clés des admins. Beryl consomme le certificat, pas la clé brute.
4. Vault avec SSH dynamique — HashiCorp Vault peut générer un certificat SSH à la volée pour chaque session. Beryl demande au vault, reçoit le cert, l'utilise, il expire.

Ce que l'architecture proposée permet déjà

- La résolution `ssh_keys: [filename]` dans un `<domaine>.yml` est déjà indirecte. Demain, beryl pourrait interpréter un identifiant spécial (ex : `yubikey://philippe.aloli.fr`) qui pointe sur un token PKCS#11 plutôt que sur un fichier local.
- Le provider `ComputeProvider` gère lui-même l'injection de la clé au rescue. L'admin fournit la référence (filename, vault path, cert), beryl ne la manipule pas directement.

Note de sécurité

Même si cet ADR ne livre pas de coffre-fort, il pose la règle : **aucune clé privée ne transite par beryl**. Beryl lit un contenu public (clé `.pub`), consomme des credentials via le `CredentialsStore`, et délègue l'authentification SSH au `ssh` du système (qui parle à l'agent / au token / au vault).

Alternatives considérées

A. Rester sur le modèle « un domaine = un fournisseur »

Rejetée. Ne couvre pas le cas Gandi pour DNS + OVH pour hébergement.

B. `.env.yml` indexé par domaine avec répétition

Rejetée. Force la duplication `aloli.net` / `aloli.fr` quand les deux sont chez le même compte ALOLI. Incohérent avec la notion de société.

C. Un seul provider par domaine + providers attachés aux hosts

Rejetée pour factorisation. Si 5 hosts Scaleway dans `aloli.net` partagent la même clé SSH Scaleway, il faut la factoriser au niveau domaine, pas répéter 5 fois.

D. ADR actuel — société × fournisseur dans `.env.yml`, dossier par société

Retenue. Modélise finement les cas réels, extensible sans régression.

Références

- Discussion 23 avril 2026 (Philippe / assistant) — synthèse des cas réels multi-société / multi-fournisseur.
- Règle Aloli « pas de défaut silencieux » — influence l'obligation de déclarer `dns_provider` et `provider` explicitement.
- Règle Aloli « convention over configuration pragmatique » — les défauts sensés restent possibles (ex : `society` auto-détectée du chemin, `--account` optionnel si une seule société).